

CI/CD no Microsoft Fabric

Guia Prático com Azure DevOps e PyFabricOps

Baseado no workshop de 3 dias conduzido por **Sidney** e **Alison** para a Comunidade Power BI Brasil — conceitos, arquitetura, boas práticas e exemplos prontos para usar.

Sumário

01 Visão Geral e Objetivo

- O que é CI/CD
- Por que usar no Fabric

02 Conceitos Fundamentais

- DevOps
- Git vs Plataformas
- Conventional Commits

03 Arquitetura de Referência

- Camadas Bronze/Silver/Gold
- Diagrama do fluxo

04 Configuração do Ambiente

- Service Principal
- Key Vault
- Integração Git

05 Fluxo de Trabalho Git

- Estrutura de branches
- Comandos essenciais
- Recuperação

06 Pipelines CI/CD

- Arquivo YAML
- Métodos de deploy
- PyFabricOps

07 Boas Práticas

- Workspaces
- Segurança
- .gitignore

08 Troubleshooting

- Erros comuns e soluções

09 Referências

- Documentação e recursos

01 - Visão Geral e Objetivo

Este ebook reúne os conceitos, templates e práticas do workshop de 3 dias sobre **CI/CD no Microsoft Fabric**. O objetivo central é capacitar times de dados a automatizar deployments, versionar artefatos e adotar cultura DevOps — independentemente das ferramentas específicas de cada empresa.

O que é CI/CD?

CI (Integração Contínua) é a prática de integrar código frequentemente ao repositório central, com validações automáticas a cada merge. **CD (Entrega Contínua)** automatiza o processo de publicação entre ambientes.

Pilar	Descrição
Pessoas	Times alinhados em processos e responsabilidades
Processos	Fluxos de revisão, aprovação e deploy padronizados
Produtos	Ferramentas: Azure DevOps, Fabric, Git, Key Vault, PyFabricOps

Por que usar no Microsoft Fabric?

- Elimina deployments manuais propensos a erro
- Garante ambientes de Dev e Prod sempre idênticos
- Permite múltiplas equipes trabalhando simultaneamente sem conflito
- Facilita rollback rápido em caso de problemas em produção
- Automatiza validação de qualidade antes de cada publicação

02 - Conceitos Fundamentais

DevOps

DevOps é o conjunto de **pessoas, processos e produtos** que habilitam a entrega contínua de valor. Não é apenas uma ferramenta — é uma cultura e uma forma de trabalhar.

Git vs Plataformas de Colaboração

	Git	Azure DevOps / GitHub / GitLab
O que é	Sistema de versionamento local	Plataforma de colaboração na nuvem
Roda onde	Na máquina do desenvolvedor	Na web
Instalação	Opcional no Fabric	Acesso via browser
Destaque	Controle total local	GitLab: suporte a redes privadas

Conventional Commits

Padrão de mensagens de commit que facilita automação de changelogs e revisão de código:

■ Padrão de Conventional Commits

```
feat:    Nova funcionalidade
fix:     Correção de bug
docs:    Atualização de documentação
refactor: Refatoração sem mudança de comportamento
test:    Adição ou correção de testes
chore:   Manutenção (dependências, CI, etc.)
```

Exemplos:

```
git commit -m "feat: adiciona pipeline de ingestão de vendas"
git commit -m "fix: corrige ID do lakehouse no workspace dev"
```


04 - Configuração do Ambiente

1. Service Principal (App Registration)

O Service Principal é a identidade usada para automação. Nunca use contas pessoais em pipelines de CI/CD.

- Acesse: Azure Portal → Azure Active Directory → App Registrations → New Registration
- Anote: Application (Client) ID, Object ID, Directory (Tenant) ID
- Em Certificates & Secrets, crie um novo secret (validade máx. 24 meses)
- Adicione o Service Principal ao Workspace do Fabric como Contributor

■ Implemente rotação automática de secrets. Nunca gerencie manualmente. Secrets expiram em até 24 meses.

2. Azure Key Vault

Armazenamento centralizado e seguro de credenciais. Custo aproximado: ~R\$ 1,00/mês.

Role	Permissão
Key Vault Secrets User	Leitura de secrets (pipelines)
Key Vault Secrets Officer	Criação e alteração de secrets (admins)
Key Vault Reader	Visualização de metadados do vault

■ Recomendação de região: use regiões mais baratas para ambientes de teste, e Brasil Sul para produção (reduz custo de transferência de dados).

3. Integração Git no Fabric Workspace

- No Workspace: Workspace Settings → Git Integration
- Conecte ao repositório Azure DevOps desejado
- Selecione a branch correspondente ao ambiente (dev para Dev, main para Prod)

■ Ambientes de produção NUNCA devem estar conectados à branch dev. Isso causaria conflitos de Logical ID e inconsistências nos artefatos.

4. Variáveis de Ambiente (Library Variables)

■ Exemplo de variáveis por ambiente

Configure no Azure DevOps: Pipelines → Library

```
LAKEHOUSE_ID:      'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx'  
DATABASE_CONNECTION: 'Server=...;Database=...'  
WORKSPACE_ID:      'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx'  
TENANT_ID:          'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx'
```

■■ NUNCA armazene senhas ou client secrets nas variáveis do Fabric. Use sempre o Key Vault.

05 - Fluxo de Trabalho Git

Estrutura de Branches

```
■ Estrutura recomendada de branches
main (produção – protegida, exige aprovação de PR)
|
+■ dev (desenvolvimento integrado)
  |
  +■ feature/ingestao-vendas
  +■ feature/modelo-financeiro
  +■ feature/pipeline-qualidade
  |
  +■ hotfix/correcao-lakehouse-id (direto para main, quando urgente)
```

Comandos Essenciais

```
■ Comandos do dia a dia
# Clonar repositório
git clone <url-do-repositorio>

# Criar e trocar para nova branch de feature
git checkout -b feature/nome-da-feature

# Verificar status das mudanças
git status

# Adicionar arquivos ao stage e commitar
git add .
git commit -m "feat: adiciona pipeline de ingestão de vendas"

# Enviar branch para o repositório remoto
git push origin feature/nome-da-feature

# Atualizar branch local com mudanças remotas
git pull origin dev
```

Recuperação de Dados

Uma das maiores vantagens do Git é a capacidade de recuperar estados anteriores. Se um desenvolvedor deletar notebooks sem commitar, abra um chamado na Microsoft — é possível recuperar via portal de administração. Para alterações comitadas:

■ Recuperação e desfazer alterações

Desfazer último commit (mantendo as mudanças locais)

```
git reset --soft HEAD~1
```

Reverter um commit específico (seguro para branches compartilhadas)

```
git revert <hash-do-commit>
```

Ver histórico de commits

```
git log --oneline
```

Trazer commit específico de outra branch (cherry-pick)

```
git cherry-pick <hash-do-commit>
```

Forçar branch local a espelhar o remoto (cuidado!)

```
git fetch origin
```

```
git reset --hard origin/dev
```

06 - Pipelines CI/CD

Estrutura do Arquivo YAML

■ azure-pipelines.yml — exemplo completo

```
trigger:
  branches:
    include:
      - main    # Deploy para prod apenas no merge para main

pr:
  branches:
    include:
      - dev     # Testes automáticos ao abrir PR para dev
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  - group: fabric-prod-variables    # Library criada no Azure DevOps

stages:
  - stage: Validate
    jobs:
      - job: RunTests
        steps:
          - script: |
              pip install pyfabricops
              python run_tests.py
              displayName: 'Executar testes de qualidade'

  - stage: Deploy
    dependsOn: Validate
    condition: succeeded()
    jobs:
      - job: DeployToProduction
        steps:
          - script: |
              python deploy.py --env prod
              displayName: 'Deploy para Producao'
```

Comparativo de Métodos de Deploy

Método	Quando usar	Limitações
Deployment Pipeline (Fabric)	Projetos simples, hot fixes manuais	Não troca params Direct Lake; não versiona Dataflow G1
Azure Pipeline (YAML)	Automação completa, troca de variáveis	Requer configuração mais elaborada
PyFabricOps / Yemo	Deploy seletivo, modelos semânticos via API	Curva de aprendizado inicial

■■ Dataflow G1 NÃO é versionável no Git e será descontinuado. Migre para Dataflow G2: melhor performance e menor consumo de capacidade.

07 - Boas Práticas

Organização de Workspaces

■ Faça	■ Evite
Um Workspace por equipe/repositório	Misturar múltiplas equipes no mesmo workspace
Dev descentralizado por área de negócio	Conectar Prod diretamente à branch dev
Prod centralizado e gerenciado pela TI	Usar funcionalidades Preview em produção
Apagar Workspaces temporários após merge	Acumular workspaces de feature sem limpeza

Segurança

- Princípio do menor privilégio em todas as permissões
- Service Principal ao invés de contas pessoais para automação
- Secrets gerenciados via Key Vault com rotação automática
- Arquivos sensíveis listados no .gitignore
- Pull Requests obrigatórios para merges em main
- Ao menos um aprovador para mudanças em produção

.gitignore Recomendado para Projetos Fabric/Power BI

```
■ .gitignore
# Variáveis de ambiente locais
.env
*.env

# Cache do Power BI Desktop
*.pbix
**/.pbi/

# Arquivos de sistema
.DS_Store
Thumbs.db

# Pastas de dependências locais
__pycache__/
*.pyc
.venv/
node_modules/
```

IDs vs Nomes de Artefatos

Sempre use **IDs** para referenciar artefatos (Lakehouse, Notebooks, etc.), nunca nomes. Nomes podem mudar entre ambientes; IDs são únicos e estáveis.

■ Referência de artefatos por ID

CORRETO: busca o ID via variável de ambiente

```
import os
```

```
lakehouse_id = os.getenv('LAKEHOUSE_ID')
```

INCORRETO: hardcoded por nome

```
lakehouse_name = 'lakehouse_producao' # pode mudar entre ambientes!
```

08 · Troubleshooting

Problema: Artefato não está sendo comitado no Fabric

Causa provável: O Service Principal não tem acesso à conexão de dados usada pelo artefato.

- Verifique se o Service Principal tem permissão na fonte de dados
- Confirme se todas as conexões do artefato estão configuradas
- Verifique as permissões no Key Vault (Secret User no mínimo)

Problema: IDs incorretos após deploy

Causa provável: IDs de Lakehouse e artefatos mudam entre workspaces.

- Após o primeiro deploy, atualize manualmente os IDs nas variáveis da Library
- No próximo deploy, o pipeline usará os IDs corretos automaticamente
- Documente o mapeamento de IDs por ambiente

Problema: Notebook deletado sem commit

Causa provável: Perda de código não versionado.

- Abra um chamado na Microsoft — possível recuperação via portal admin
- Se havia commit anterior: git revert ou git reset para restaurar
- Prevenção: sincronize sempre com Git antes de deletar qualquer artefato

Problema: Pipeline não executa ao abrir PR

Causa provável: Trigger de PR não está configurado no YAML.

- Adicione a seção pr: no início do azure-pipelines.yml
- Inclua as branches dev e main no include
- Salve e commite o YAML corrigido

Problema: Erro de conexão no Key Vault

Causa provável: Permissões insuficientes ou região incorreta.

- Verifique o role atribuído ao Service Principal (mínimo: Secret User)
- Confirme que o Key Vault e o Fabric estão no mesmo tenant
- Revise as políticas de acesso no portal Azure

09 · Referências e Recursos

Documentação Oficial

- Microsoft Fabric: learn.microsoft.com/fabric
- Azure DevOps Pipelines: learn.microsoft.com/azure/devops/pipelines
- Azure Key Vault: learn.microsoft.com/azure/key-vault
- Git Integration no Fabric: learn.microsoft.com/fabric/cicd

Ferramentas Utilizadas no Workshop

Ferramenta	Uso	Onde encontrar
PyFabricOps (Yemo)	Deploy de artefatos via API do Fabric	Biblioteca open source — Alison
Azure DevOps	Repositório, Pipelines e Library de variáveis	dev.azure.com
Azure Key Vault	Gestão segura de secrets e credenciais	portal.azure.com
VS Code	Desenvolvimento e trabalho local com Git	code.visualstudio.com
GitHub Copilot	Revisão e geração de código no repositório	github.com/features/copilot

Convenções de Nomenclatura

Artefato	Padrão	Exemplo
Repositório	minusculas_com_underscore	engenharia_dados_vendas
Branch de feature	feature/descricao-curta	feature/pipeline-nfe
Branch de hotfix	hotfix/descricao-curta	hotfix/correcao-lakehouse-id
Workspace Dev	[Equipe] - Dev	Engenharia - Dev
Workspace Prod	[Equipe] - Prod	Engenharia - Prod